useCallback Hook Explained

1. The core idea: functions are re-created on every render

Every time a component re-renders, the whole function body runs again, so every function declared inside it is **created fresh**. In JavaScript, two functions with identical code are still different objects:

```
const a = () => {};
const b = () => {};
console.log(a === b); // false (different references in memory)
```

This is called **referential equality**. React compares props using it.

Your `ChildComp` is wrapped in `React.memo`. `React.memo` tells React: “Skip re-rendering this component if its props have not changed.” It checks this with a **shallow comparison**, which compares primitives by value and functions and objects by reference.

So there is a trap. If the parent passes a function as a prop and that function is re-created on every render, `React.memo` sees a “new” prop every time, and the memoization is useless.

`useCallback` fixes this by returning the **same function reference** between renders, until its dependencies change.

2. Your example, without useCallback

App.jsx

```
import { useState } from "react";
import ChildComp from "./components/ChildComp";

function App() {
  const [count, setCount] = useState(0);
  const [other, setOther] = useState(0);

  // new function created on EVERY render of App
  const handleClick = () => {
    console.log("Child button clicked");
  };

  return (
    <div>
      <p>count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment count</button>

      <p>other: {other}</p>
      <button onClick={() => setOther(other + 1)}>Increment other</button>

      <ChildComp buttonName="Click Me" handleClick={handleClick} />
    </div>
  );
}

export default App;
```

What happens: click “Increment count” and `App` re-renders. `handleClick` is created again as a new function, so `React.memo` sees `props.handleClick` as changed, and the console prints `child component got re-rendered`. This happens even though the child doesn’t use `count` or `other` at all, and the same happens on every click.

3. Same example, with useCallback

```
import { useState, useCallback } from "react";
import ChildComp from "./components/ChildComp";

function App() {
  const [count, setCount] = useState(0);
  const [other, setOther] = useState(0);

  // same function reference is reused between renders
  const handleClick = useCallback(() => {
    console.log("Child button clicked");
  }, []);

  return (
    <div>
      <p>count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment count</button>

      <p>other: {other}</p>
      <button onClick={() => setOther(other + 1)}>Increment other</button>

      <ChildComp buttonName="Click Me" handleClick={handleClick} />
    </div>
  );
}

export default App;
```

What happens: `App` still re-renders on every click, but `handleClick` keeps the same reference. All of the child's props are unchanged, so `React.memo` skips the child. The log prints **only once**, on the first render.

Scenario	handleClick reference	Child re-renders on parent update?
Without `useCallback`	New every render	Yes, `React.memo` is defeated
With `useCallback` + `React.memo`	Same between renders	No, child is skipped
With `useCallback` but **no** `React.memo`	Same	Yes, a plain child always re-renders with its parent

The third row matters. `useCallback` **alone does nothing visible. It only pays off together with something that compares references.**

4. The dependency array

```
const handleClick = useCallback(() => {
  // code
}, [dep1, dep2]);
```

React returns the cached function until one of the dependencies changes, then it creates a new one.

Stale closure problem: a function “remembers” the variable values from the render in which it was created.

```
// BUG: count is always 0 inside this function
const logCount = useCallback(() => {
  console.log(count);
}, []); // count is missing from the dependency array

// Fix 1: add the dependency (function is re-created when count changes)
const logCount = useCallback(() => {
  console.log(count);
}, [count]);
```

When you only need to update state, use the functional updater and keep the dependency array empty:

```
// Fix 2: no dependency on count, and the function stays stable
const increment = useCallback(() => {
  setCount((prev) => prev + 1);
}, []);
```

5. Second example: a todo list with delete

This is where `useCallback` matters in real apps, because a list renders many children.

TodoItem.jsx

```
import React from "react";

const TodoItem = React.memo(({ todo, onDelete }) => {
  console.log("Rendering:", todo.text);
  return (
    <li>
      {todo.text}
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </li>
  );
});

export default TodoItem;
```

TodoApp.jsx

```
import { useState, useCallback } from "react";
import TodoItem from "./TodoItem";

function TodoApp() {
  const [todos, setTodos] = useState([
    { id: 1, text: "Learn React" },
    { id: 2, text: "Learn useCallback" },
    { id: 3, text: "Build a project" },
  ]);
  const [input, setInput] = useState("");

  const handleDelete = useCallback((id) => {
    setTodos((prev) => prev.filter((t) => t.id !== id));
  }, []);

  return (
    <div>
      <input value={input} onChange={(e) => setInput(e.target.value)} />
      <ul>
        {todos.map((todo) => (
          <TodoItem key={todo.id} todo={todo} onDelete={handleDelete} />
        ))}
      </ul>
    </div>
  );
}

export default TodoApp;
```

Without `useCallback`**:** every keystroke in the input re-renders `TodoApp`, creates a new `handleDelete`, and re-renders **all** `TodoItem`s. With 1000 items, typing becomes laggy.

With `useCallback`**:** typing does not re-render any `TodoItem`. When you delete one item, only the list changes, and the remaining items skip rendering because their `todo` and `onDelete` props are unchanged.

6. When to use useCallback

Use it when:

- **Passing a function to a child wrapped in** `React.memo` (your case).
- **The function is a dependency of** `useEffect`**,** `useMemo`**, or another hook**, so it doesn't cause the effect to re-run on every render.
- **Passing callbacks into large lists** or expensive-to-render components.
- **Custom hooks** that return functions, so consumers get stable references.

Don't use it when:

- The child is **not** memoized, because there is nothing to benefit.

- The function is only used in the same component (like a plain `onClick` on a `<button>`). Passing a new function to a DOM element costs almost nothing.
- You add it “just in case”. `useCallback` has its own cost (storing the function and comparing dependencies), so overusing it can make things slower and the code harder to read.

Optimize after you measure a real performance problem (React DevTools Profiler), not before.

7. Interview questions and answers

Q1. What is `useCallback`?

A hook that memoizes a function and returns the same reference between renders, unless its dependencies change. Signature: `useCallback(fn, deps)`.

Q2. Why do we need it if functions already work normally?

Functions are re-created on every render and have a new reference each time. When passed as props to memoized children or used in hook dependency arrays, the new reference causes unnecessary re-renders or effect re-runs. `useCallback` keeps the reference stable.

Q3. What is the difference between `useCallback` and `useMemo`?

`useCallback(fn, deps)` caches the **function itself**. `useMemo(() => value, deps)` caches the **result of calling** a function. Rule of thumb: `useCallback(fn, deps)` is equivalent to `useMemo(() => fn, deps)`.

Q4. Does `useCallback` prevent a function from running?

No. It only controls whether the function **reference** changes. It doesn’t stop the function from being called. To avoid recomputing a value, use `useMemo`.

Q5. Does `useCallback` work without `React.memo`?

It runs without errors, but gives no rendering benefit for a plain child, because a non-memoized child re-renders whenever its parent does, regardless of props.

Q6. What does `React.memo` do and how does it compare props?

It is a higher-order component that skips re-rendering if props are shallowly equal. Primitives are compared by value; objects, arrays, and functions are compared by reference.

Q7. What happens if you pass an empty dependency array `[]` but use state inside the function?

You get a **stale closure**: the function keeps seeing the state from the first render. Fix it by adding the dependency or using the functional updater form (`setX(prev => ...)`).

Q8. When should you avoid `useCallback`?

When the child isn’t memoized, when the function isn’t passed down or used in a dependency array, or when the component is cheap to render. Premature optimization adds complexity without benefit.

Q9. How would you debug unnecessary re-renders?

Use the React DevTools Profiler ("Highlight updates" and "Why did this render?"), or add `console.log` in the child. Then check for props that change reference every render.

Q10. Is `useCallback` a guarantee that the function is cached forever?

No. It's a performance hint, not a semantic guarantee. React may discard the cache in some situations (for example, in future features that preserve state differently), so your code should never depend on it for correctness, only for optimization.

8. Quick summary

- Functions get a **new reference on every render**.
- `React.memo` compares props by **reference**, so a new function means a re-render.
- `useCallback` keeps the **same reference** until dependencies change.
- It only helps when paired with `React.memo`, or when the function is a hook dependency.
- Use `useMemo` for caching **values**, `useCallback` for caching **functions**.